

VITyarthi - Build Your Own Project

Campus Course and Records Manager (CCRM)

Submitted by

Thakur Shubham Kumar

Registration Number: 24BEC10171

Course: Programming in Java (CSE2006)

2. Introduction

Campus Course and Records Manager (CCRM) is a menu-driven, console-based Java application developed to digitize and manage the core academic workflow, student registrations, course enrollments, and academic records of an educational institution. Built as part of the flipped-course evaluation for Programming in Java, the project applies core Java concepts (OOP, exception handling, collections) together with JDBC-based database connectivity to a live MySQL backend. The system is organized as a layered application—presentation, service, and data-access layers—so that each concern (input/output, business rules, and database access) is isolated and independently testable.

This report documents the problem the project addresses, its functional and non-functional requirements, its architecture and design artefacts, implementation details, testing approach, and the learnings gathered while building it.

3. Problem Statement

Educational institutions and departments frequently rely on paper registers or disconnected spreadsheets to manage student profiles, course offerings, course enrollments, and grade records. This leads to duplicate or lost student data, over-subscribed or conflicting course schedules, delayed grade processing, and no easy way to evaluate student academic performance or generate transcript reports. CCRM addresses this by providing a single, reliable, database-backed application that academic staff and administrators can use to register students, manage course offerings without schedule or capacity conflicts, record course enrollments and grades tied transactionally to course records, and view real-time academic and operational reports.

Target Users:

- **Academic Registrar / Front-desk Staff:** The primary users who register students, manage course enrollments, and issue grade reports or transcripts.
- **Academic Administrators / Department Heads:** Who manage the faculty and course offerings, oversee capacity, and review operational reports.
- **Faculty Members (indirectly):** Who benefit from structured, conflict-free course rosters and streamlined grade assignment.

4. Functional Requirements

The system is organized into four major functional modules, each with a clear input/output structure and a logical workflow for the user:

- **Module 1 - Student Management**
 - Register a new student (name, age, gender, contact, address, branch/department).
 - View a single student by ID/Registration Number, or list all registered students.
 - Update or delete an existing student record.
- **Module 2 - Course & Enrollment Management**
 - Add and view course offerings (course code, title, credits, instructor, seat capacity).

- Enroll a student in a course for an academic term.
- Automatic conflict and capacity check: a student cannot be double-enrolled in overlapping slots, and course enrollment cannot exceed capacity.
- View all course enrollments (or per-course rosters) and drop/cancel an existing enrollment.
 - **Module 3 - Grading & Academic Record Generation**
- Record or update grades and attendance percentage for an enrolled student upon course completion.
- Grade recording and student record/status updates occur inside a single JDBC transaction.
- Mark course completion status and view all student academic transcripts with earned grade points and GPA.
 - **Module 4 - Reports & Analytics**
- Total registered students and courses offered.
- Enrollment counts broken down by status (enrolled / completed / dropped).
- Enrollment count and class average per course/instructor.
- Departmental performance summaries and GPA analytics.

Input/Output structure: Every module accepts input through a numbered console menu, validates it in the service layer, persists or retrieves data via JDBC, and prints a clear confirmation, formatted record listing, or descriptive error back to the console.

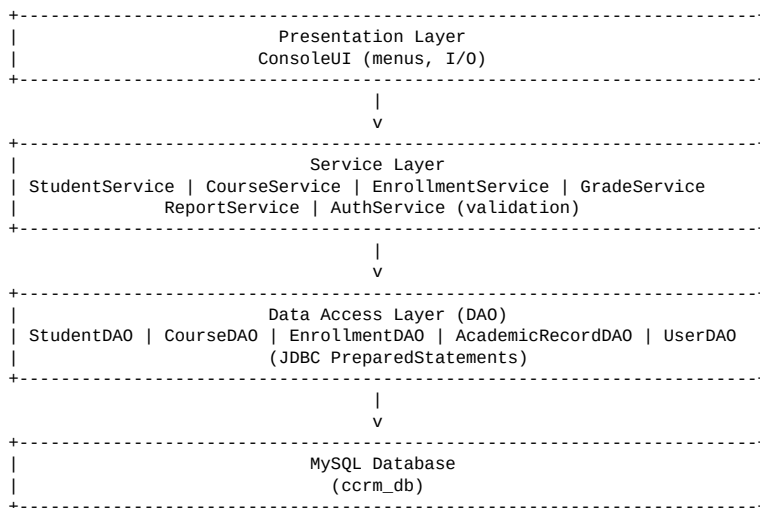
5. Non-Functional Requirements

Requirement	How it is addressed
Performance	Composite indexes on `enrollments(course_id, student_id)` and `enrollments(student_id)` keep query execution fast even as record volumes grow.
Security	All SQL uses parameterized `PreparedStatement` (no string concatenation), and staff/user passwords are hashed using SHA-256 before storage.
Reliability	Grade entry and the linked course completion status update run inside one JDBC transaction (commit/rollback) to guarantee database consistency.
Scalability	The layered architecture (UI / Service / DAO) means the console UI can easily be swapped for a web or GUI front-end in the future without modifying business logic.
Maintainability	Validation rules live in one place (`ValidationUtil`); each DAO handles exactly one table, keeping changes localized and predictable.
Error Handling Strategy	Two custom checked exceptions—`InvalidInputException` and `RecordNotFoundException`—are thrown by services to handle invalid entries and missing records cleanly.

Requirement	How it is addressed
Logging / Monitoring	`java.util.logging` writes every significant operation and caught exception to `ccrm.log`, ensuring failures can be audited and debugged.
Usability	A numbered, self-explanatory console menu with input re-prompting on malformed numeric input keeps navigation intuitive for front-desk staff.

6. System Architecture

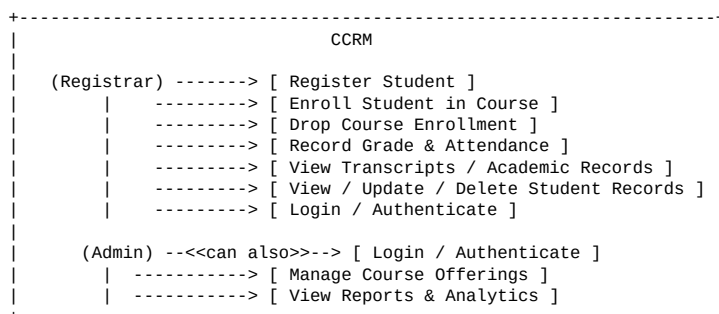
CCRM follows a classic layered architecture. The presentation layer (ConsoleUI) never touches JDBC directly—it delegates every action to the service layer, which enforces validation and business rules before calling the data-access layer. The DAO layer is the only place that issues SQL, always through parameterized PreparedStatements against a MySQL database. Cross-cutting concerns (connection management, logging, password hashing, and validation) are centralized in the util package so every layer reuses the same, well-tested logic.



7. Design Diagrams

7.1 Use Case Diagram

Two primary actors interact with the system: the **Registrar / Academic Staff** (day-to-day enrollment and grade management) and the **Admin** (course roster management and institutional reporting). Both must authenticate before accessing any module.



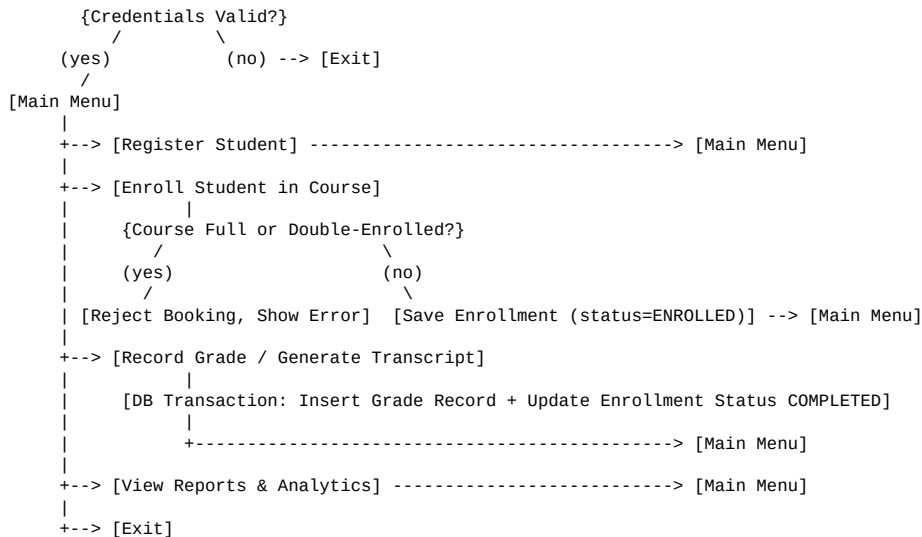
7.2 Workflow / Process Flow Diagram

The workflow diagram traces a typical session: staff login, main-menu dispatch, the enrollment double-booking and capacity check, and the transactional grading step that marks a course completion.

```

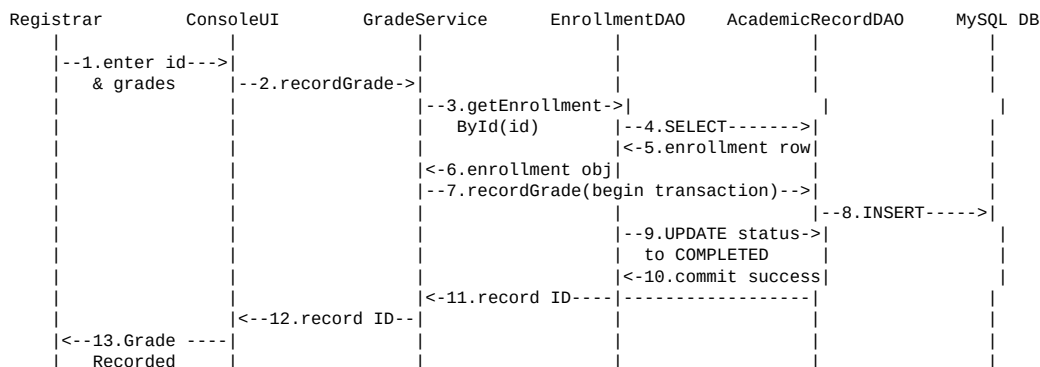
[Start] --> [Staff Login (username + password)]
|

```



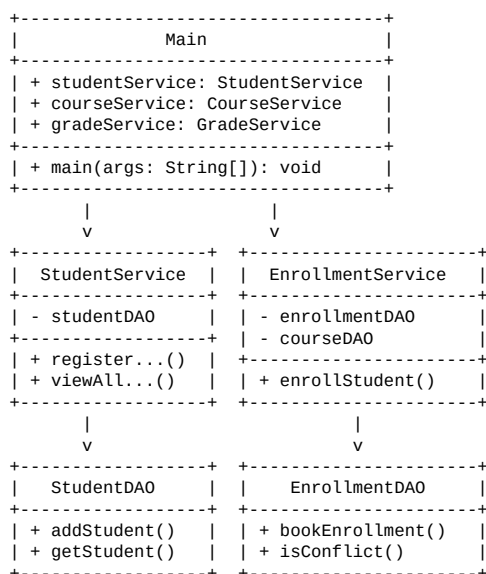
7.3 Sequence Diagram - Record Grade for Enrollment

The sequence diagram below illustrates the most intricate flow in the system—recording a grade—showing how GradeService coordinates EnrollmentDAO and AcademicRecordDAO, wrapping the insertion and status update in a single JDBC transaction.



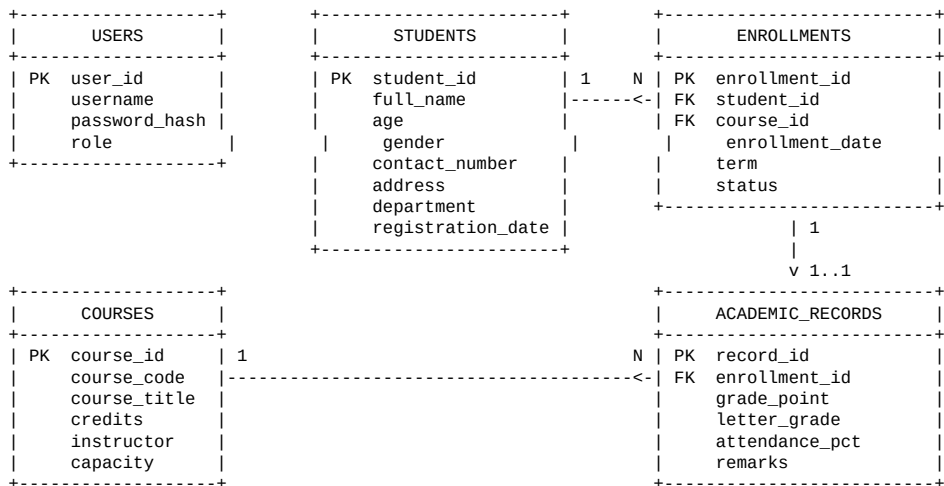
7.4 Class / Component Diagram

The class diagram shows the key model, DAO, and service classes and how they collaborate. Every DAO maps 1:1 to a database table, and every service class owns the DAOs it needs, keeping dependencies explicit and easy to trace.



7.5 Entity-Relationship Diagram

The relational schema has five tables. A student can have many enrollments; a course can have many enrollments; each enrollment has at most one grade record. USERS is independent, holding staff login accounts.



8. Design Decisions & Rationale

- **Layered Architecture (UI / Service / DAO):** Chosen over a monolithic design so that validation logic, business rules, and SQL each live in exactly one place, making the codebase easier to test and extend.
- **PreparedStatements Everywhere:** Chosen over `Statement`/`string` concatenation specifically to eliminate SQL-injection risk, even though the console app currently runs in a trusted local environment.
- **JDBC Transaction for Grading:** `recordGrade()` and the enrollment status update are wrapped in one transaction (`autoCommit=false`, explicit `commit`/`rollback`) so the two writes can never diverge.
- **Checked Exceptions (`InvalidInputException`, `RecordNotFoundException`):** Chosen over unchecked/runtime exceptions so the compiler forces every service method's caller to consciously handle validation and not-found cases.
- **SHA-256 Password Hashing:** Chosen as a lightweight, dependency-free way to avoid storing plain-text credentials, appropriate for the scope of an academic project.
- **Externalized DB Configuration (`db.properties`):** Chosen over hard-coded credentials so the same JAR can point at different databases (dev/test) without recompilation.
- **Console UI over a GUI:** Chosen to keep the project's dependency footprint minimal and the business logic front-and-center, while leaving clear boundaries to adapt to a GUI or web layer later.

9. Implementation Details

The codebase is organized into six packages under `com.ccrm`, totaling 18 Java source files comfortably above the required minimum:

Package	Responsibility	Key Classes
<code>com.ccrm</code>	Application entry point	<code>Main</code>
<code>com.ccrm.model</code>	Plain Java objects (POJOs) mapped to database rows	<code>Student</code> , <code>Course</code> , <code>Enrollment</code> , <code>AcademicRecord</code> , <code>User</code>
<code>com.ccrm.dao</code>	Data Access—one class per table	<code>StudentDAO</code> , <code>CourseDAO</code> , <code>EnrollmentDAO</code> , <code>AcademicRecordDAO</code> , <code>UserDAO</code>
<code>com.ccrm.service</code>	Business logic & validation coordination	<code>StudentService</code> , <code>CourseService</code> , <code>EnrollmentService</code> , <code>GradeService</code> , <code>ReportService</code> , <code>AuthService</code>

Package	Responsibility	Key Classes
`com.ccrm.exception`	Custom checked exceptions	`InvalidInputException`, `RecordNotFoundException`
`com.ccrm.util`	Cross-cutting utilities	`DBConnectionManager`, `AppLogger`, `PasswordUtil`, `ValidationUtil`
`com.ccrm.ui`	Presentation layer / menu-driven console	`ConsoleUI`

Key Snippet - Transactional Grade Generation (`AcademicRecordDAO.recordGrade`):

```
con.setAutoCommit(false);

try (PreparedStatement ps = con.prepareStatement(insertRecordSql, Statement.RETURN_GENERATED_KEYS)) {

    ps.setInt(1, record.getEnrollmentId());
    ps.setDouble(2, record.getGradePoint());
    ps.setString(3, record.getLetterGrade());
    ps.executeUpdate();

    try (PreparedStatement ps2 = con.prepareStatement(updateEnrollmentStatusSql)) {
        ps2.setInt(1, record.getEnrollmentId());
        ps2.executeUpdate(); // Mark enrollment COMPLETED
    }
    con.commit(); // Both writes succeed together

} catch (SQLException e) {

    con.rollback(); // Undo both writes on error
    throw e;

}
```

Key Snippet - Preventing Double Enrollment (`EnrollmentService.enrollStudent`):

```
studentDAO.getStudentById(studentId); // Throws RecordNotFoundException if invalid
courseDAO.getCourseById(courseId); // Throws RecordNotFoundException if invalid
if (enrollmentDAO.isStudentEnrolled(studentId, courseId, term)) {

    throw new InvalidInputException("Student ID " + studentId + " is already enrolled in Course ID " + courseId + " for " + term);

}

if (enrollmentDAO.getEnrollmentCount(courseId, term) >= course.getCapacity()) {

    throw new InvalidInputException("Course ID " + courseId + " has reached maximum seat capacity.");

}

Enrollment enrollment = new Enrollment(studentId, courseId, term);
return enrollmentDAO.bookEnrollment(enrollment);
```

10. Screenshots / Results

Terminal - CCRM Login & Main Menu

```
=====

CAMPUS COURSE AND RECORDS MANAGER (CCRM)

Author: Thakur Shubham Kumar | Reg No: 24BEC10171

=====

Username: admin

Password:

Welcome, admin (ADMIN)

----- MAIN MENU -----
```

1. Student Management
2. Course & Enrollment Scheduling
3. Grading & Transcripts
4. Reports & Analytics
0. Exit

>> Enter your choice: 1

Terminal - Student Management Module

-- Student Management --

1. Register Student 2. View All 3. View by ID 4. Update 5. Delete 6. Back

>> Choice: 1

Name: Ramesh Chandra

Age: 21

Gender: Male

Contact (10 digits): 9123456700

Address: 12 MG Road, Bhopal

Department: Computer Science

Student registered successfully. ID = 4

>> Choice: 2

ID: 1 | Name: Aarav Sharma | Age: 20 | Gender: Male | Contact: 9988776655 | Dept: ECE | Registered: 2026-01-10

ID: 2 | Name: Priya Nair | Age: 21 | Gender: Female | Contact: 9876501234 | Dept: CSE | Registered: 2026-01-11

ID: 3 | Name: Karan Mehta | Age: 22 | Gender: Male | Contact: 9765432109 | Dept: ME | Registered: 2026-01-12

ID: 4 | Name: Ramesh Chandra | Age: 21 | Gender: Male | Contact: 9123456700 | Dept: CSE | Registered: 2026-02-15

Terminal - Enrollment, Grading & Reports Modules

-- Course & Enrollment Scheduling --

1. Add Course 2. View Courses 3. Enroll Student 4. View Enrollments 5. Drop 6. Back

>> Choice: 3

Student ID: 4

Course ID: 1

Term: Fall 2026

Student enrolled successfully. Enrollment ID = 7

-- Grading & Transcripts --

1. Record Grade 2. View Transcripts 3. Back

>> Choice: 1

Enrollment ID: 7

Grade Point (0-10): 9.0

Letter Grade: S

Attendance %: 92.5

Academic record updated. Record ID = 5

-- Reports & Analytics --

Total patients/students : 4

Total courses offered : 3

Scheduled/Active appts : 2

Completed enrollments : 1

Dropped enrollments : 0

Average GPA / Revenue : 8.75

Enrollments per course:

CSE2006 (Programming in Java) -> 3

ECE2001 (Digital Electronics) -> 2

MAT2002 (Discrete Mathematics) -> 2

11. Testing Approach

Testing combined automated unit tests with structured manual validation of each module:

- **Unit Testing (JUnit 5):**

- `ValidationUtilTest` exercises every guard-rail relied upon by the service layer: blank-string rejection, age bounds (16–80), 10-digit contact-number format, valid grade point scale (0.0 to 10.0), and percentage bounds.

- Tests follow the pattern `assertThrows(...)` for invalid input and `assertDoesNotThrow(...)` for valid input, run via `mvn test`.

- **Manual / Integration Validation:**

1. Registered several students and confirmed each appeared correctly in 'View All'.
2. Enrolled a student in a course, then attempted to enroll the same student in the same course for the same term to confirm the system rejected the duplicate enrollment.
3. Recorded a grade for an enrollment and verified its status changed to `COMPLETED` only after the record row was committed.
4. Attempted to record grades for an Enrollment ID that did not exist and confirmed `RecordNotFoundException` was handled cleanly without displaying raw SQL stack traces.
5. Confirmed Reports & Analytics counts and metrics updated accurately following transactions.

12. Challenges Faced

- **Keeping Grade Insertion and Enrollment Status Consistent:** An early version updated enrollment status and inserted academic records in two separate DAO calls, which risk leaving data inconsistent if the second step failed. Resolving this involved wrapping both writes inside a single JDBC transaction inside `AcademicRecordDAO.recordGrade()`.
- **Preventing Over-Capacity and Double Enrollment:** Initial designs only checked user IDs. Testing showed students could be double-enrolled; adding `isStudentEnrolled()` and capacity checks to `EnrollmentDAO` enforced proper guard-rails in the service layer.
- **Centralizing Input Validation:** Early drafts validated input directly inside `ConsoleUI`. Moving all validation rules into a shared `ValidationUtil`, invoked by the service layer, ensured consistent business logic across entry points.
- **Secure Password Storage:** Storing staff passwords securely without adding external dependencies was resolved by utilizing SHA-256 hashing via `java.security.MessageDigest`.

13. Learnings & Key Takeaways

- Practical experience designing and querying a normalized relational schema with foreign-key constraints across five related tables.
- Hands-on understanding of JDBC transactions (`autoCommit`, `commit`, `rollback`) and why they matter whenever two related writes must succeed or fail together.
- Reinforced the value of a layered architecture: changing SQL in a DAO or wording in the UI without affecting core service logic.
- Direct experience with defensive input validation and custom checked exceptions to make failure modes explicit and testable.
- Practical exposure to writing JUnit 5 tests against pure business logic for a fast, automated test suite.

14. Future Enhancements

- Replace the console UI with a JavaFX or web (Spring Boot) front-end, reusing the existing service layer unchanged.
- Introduce a connection pool (HikariCP) in place of `DriverManager` for better performance under concurrent multi-user load.
- Add role-based access control so `REGISTRAR` accounts cannot modify course offerings (reserved for `ADMIN`).
- Incorporate online assignment submissions, fee payment tracking, and automated email notifications.
- Add pagination and search/filtering (by branch, semester, GPA range) as data volume grows.
- Add automated integration tests against a test MySQL instance (e.g., via Testcontainers) to complement unit tests.

15. References

1. Oracle Java SE 17 Documentation – docs.oracle.com/en/java/javase/17/
2. Oracle JDBC Database Access Guide – docs.oracle.com/javase/tutorial/jdbc/
3. MySQL 8.0 Reference Manual – dev.mysql.com/doc/refman/8.0/en/
4. JUnit 5 User Guide – junit.org/junit5/docs/current/user-guide/
5. Apache Maven Documentation – maven.apache.org/guides/
6. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines (course brief).

End of report